

// TOPIC: STL Introduction In C++

// NOTES:

// 1. STL: Standard Template Library

// 2. It consist of three components

// a. Container

// b. Iterator

// c. Algorithm

std array C++

// TOPIC: std::array

// Syntax: std::array<T, N> array;

// NOTICE:

// 1. std::array is a container that encapsulates fixed size arrays.

// 2. arraysize is needed at compile time.

// 3. Assign by value is actually by value.

// 4. Access Elements:

// a. at()

// b. []

// c. front()

// d. back()

// e. data() // gives access to the underlying array

#include <iostream>

#include <array>

using namespace std;

```
int main() {  
  
    // Declare  
    std::array<int, 5> myarray;  
  
    // Initialization  
    std::array<int, 5> myarray = {1,2,3,4,5}; // initializer list  
    std::array<int, 5> myarray2 {1,2,3,4,5}; // uniform initialization  
  
    // Assign using initializer list  
    std::array<int, 5> myarray;  
    myarray = {1,2,3,4,5};  
  
    return 0;  
}
```


Vector In C++

// TOPIC: std::vector

// NOTES:

// 1. std::vector is a sequence container and also known as Dynamic Array or Array List.

// 2. Its size can grow and shrink dynamically, and no need to provide size at compile time.

```
// ELEMENT ACCESS
```

```
// at(), [], front(), back(), data()
```

```
// MODIFIERS:
```

```
// insert(), emplace(), push_back(), emplace_back(), pop_back(), resize(), swap(),  
erase(), clear()
```

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main() {
```

```
    // Declarations
```

```
    std::vector<int> arr1;
```

```
    std::vector<int> arr2 (5, 20);
```

```
    std::vector<int> arr3 = { 1, 2, 3, 4, 5 }; // initializer list
```

```
    std::vector<int> arr4 { 1, 2, 3, 4, 5 }; // uniform initialization
```

```
    // Accessing Elements
```

```
    arr2[3] = 10;
```

```
    arr2.at(3) = 10;
```

```
    for(int i=0; i<=10; ++i) { arr1.push_back(i); }
```

```
    for(const auto& vec: arr1) { cout << vec << endl; }
```

```
    return 0;
```

```
}
```

// TOPIC: Best way to use vector in C++

// PROBLEM IN VECTOR ?

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main() {
```

```
    vector<int> Vec;
```

```
    Vec.reserve(31);
```

```
    for(int i=0; i < 2; ++i) {
```

```
        Vec.push_back(i);
```

```
        cout << "Size:" << Vec.size() << " Capacity:" << Vec.capacity() << endl;
```

```
    }
```

```
    return 0;
```

```
}
```

// TOPIC: std::set<T>

// SYNTAX: std::set<T> obectName;

// 1. std::set is an Associative Container that contains a sorted set of unique objects of type Key.

// 2. It is usually implemented using Red-Black Tree.

// 3. Insertion, Removal, Search have logarithmic complexity.

// 4. If we want to store user defined data type in set then we will have to provide
// compare function so that set can store them in sorted order.

// 5. We can pass the order of sorting while constructing set object.

// BOTTOM LINE:

// It store unique elements and they are stored in sorted order (A/D)

```
#include <iostream>
```

```
#include <set>
```

```
#include <string>
```

```
#include <functional>
```

```
using namespace std;
```

```
class Person{
```

```
    public:
```

```
        float age;
```

```
        string name;
```

```
        bool operator < (const Person& rhs) const { return age<rhs.age; }
```

```
        bool operator > (const Person& rhs) const { return age>rhs.age; }
```

```
};
```

```
int main() {
```

```
    set<Person, std::less<>> Set = {{30, "Rupesh"}, {25, "Hitesh"}, {22, "Jhon"}};
```

```
    for(const auto& e: Set){
```

```
        cout << e.age << " " << e.name << endl;
```

```
    }
```

```
    return 0;
```

```
}
```

```
-----  
-----
```

// TOPIC: Container Inside Container

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    std::vector<vector<int>> Tree;
```

```
    int edge, n1, n2;
```

```
    cin >> edge;
```

```
    Tree.resize(edge);
```

```
    for (int i = 0; i < edge; ++i) {
```

```
        cin >> n1 >> n2;
```

```
        Tree[n1].push_back(n2);
```

```
    }
```

```
    for (const auto &e1: Tree) {
```

```
        for (const auto &e2: e1) {
```

```
            cout << e2;
```

```
        }
```

```
        cout << endl;
    }

    return 0;
}
```


// TOPIC: std::multiset<T>

// SYNTAX: std::multiset<T> objectName;

// 1. std::multiset is an Associative Container that contains a sorted set of duplicate objects of type Key.

// 2. It is usually implemented using Red-Black Tree.

// 3. Insertion, Removal, Search have logarithmic complexity.

// 4. If we want to store user defined data type in multiset then we will have to provide

// compare function so that multiset can store them in sorted order.

// 5. We can pass the order of sorting while constructing set object.

// BOTTOM LINE:

// multiset is similar to set except it can have multiple elements with same value.

```
#include <iostream>
```

```
#include <set>
```

```
#include <string>
```

```
#include <functional>
```

```
using namespace std;
```

// Example

```
// int main() {  
//     std::multiset<int, std::greater<int>> multiSet = {5,2,4,3,2,5};  
//     multiSet.insert(20);  
//     for(const auto& e: multiSet) {  
//         cout << e << endl;  
//     }  
// }
```

//Example : User Defined

```
class Person{  
    public:  
        float age;  
        string name;  
        bool operator < (const Person& rhs) const { return age<rhs.age; }  
        bool operator > (const Person& rhs) const { return age>rhs.age; }  
};
```

```
int main() {  
    std::multiset<Person, std::greater<>> multiSet = {{25, "Rupesh"}, {20, "Hitesh"}};  
  
    for(const auto& e: multiSet){  
        cout << e.age << " " << e.name << endl;  
    }  
    return 0;  
}
```

// TOPIC: Map In C++

// NOTES:

// 1. Syntax: map<T1, T2> obj; // where T1 is key type and T2 is value type.

// 2. std::map is associative container that store elements in key value combination

// where key should be unique, otherwise it overrides the previous value.

// 3. It is implement using Self-Balance Binary Search Tree (AVL/Red Black Tree) .

// 4. It store key value pair in sorted order on the basis of key (assending/decending).

// 5. std::map is generally used in Dictionay type problems.

// EXAMPLE: Dictionary

#include <iostream>

#include <map>

#include <functional>

#include <vector>

using namespace std;

class Person{

public:

float age;

string name;

bool operator < (const Person& rhs) const { return age<rhs.age; }

bool operator > (const Person& rhs) const { return age>rhs.age; }

};

```

int main() {
    std::map<Person, int, std::less<>> Map;
    Person p1, p2;
    Map[p1] = 1;
    Map[p2] = 2;

    cout << Map[p1] << endl;
    cout << Map[p2] << endl;
    return 0;
}

```


// TOPIC: Multimap In C++

// SYNTAX: multimap<T1, T2> obj; // where T1 is key type and T2 is value type.

// NOTES:

// 1. Multimap is an associative container that contains a sorted list of key-value pairs,

// while permitting multiple entries with the same key.

// 2. It store key value pair in sorted order on the basis of key (ascending/decending).

// Syntax: std::multimap<char, int, std::greater<>> Multimap;

// 3. Data structure used in multimap is not defined by standard, but red-black tree is assumed by most of the people.

// 4. Lookup: count, find, contains, equal_range, lower_bound, upper_bound

// 5. We dont have at() and [] functions to get element like we had in std::map.

#include <iostream>

```
#include <vector>
```

```
#include <map>
```

```
using namespace std;
```

```
int main() {
```

```
    std::multimap<char, int> Multimap;
```

```
    Multimap.insert(make_pair('a', 1));
```

```
    Multimap.insert(make_pair('a', 2));
```

```
    Multimap.insert(make_pair('a', 3));
```

```
    Multimap.insert(make_pair('b', 4));
```

```
    Multimap.insert(make_pair('b', 5));
```

```
    // auto pair = Multimap.find('a');
```

```
    // cout << pair->first << " " << pair->second << endl;
```

```
    // // Iterate over multimap
```

```
    // for(auto& elm: Multimap){
```

```
        // cout << elm.first << " " << elm.second << endl;
```

```
    // }
```

```
    // cout << endl;
```

```
    // // Get all the pairs of given key
```

```
    auto range = Multimap.lower_bound('b');
```

```
    cout << range->first << " " << range->second << endl;
```

```
    return 0;
```

```
}
```

// TOPIC: Emplace In STL

// 1. All the containers supports insert and emplace operation to store data.

// 2. Emplace is used to construct object in-place and avoids unnecessary copy of objects.

// 3. Insert and Emplace is equal for premutive data types but when we deal with heavy objects

// we should use emplace if we can for efficiency.

```
#include <iostream>
```

```
#include <set>
```

```
using namespace std;
```

```
class A {
```

```
    public:
```

```
    int x;
```

```
    A(int x=0): x{x} { cout << "Construct" << endl; };
```

```
    A(const A& rhs) { x = rhs.x; cout << "Copy" << endl; }
```

```
};
```

```
bool operator < (const A& lhs, const A& rhs) { return lhs.x < rhs.x; }
```

```
int main() {
```

```
    set<A> Set;
```

```
    // A a(10);
```

```
    //Set.insert(A(10));
```

```
    //Set.emplace(10);
```

```

    for ( auto & elm: Set){
        cout << elm.x << endl;
    }
    return 0;
}

```


// TOPIC: forward_list In STL

// 1. This is single linked list what we know from C programming language.

// 2. Why forward_list why not single_list ?

// 3. We should use this class instead of traditional single linked list because

// a. Well tested

// b. Bunch of available function

// 4. Few Available Operations

// operator =, assign, front, empty, max_size, clear, insert_after, emplace_after,
 reverse, sort, merge

// splice_after, unique, remove, remove_if, resize

#include <iostream>

#include <forward_list>

using namespace std;

int main()

{

forward_list<int> list1 = {5, 2, 4, 6, 2};

forward_list<int> list2 = {7, 6, 1, 9};

list1.resize(6);

```

for(auto& elm: list1)
    cout << elm << endl;

return 0;
}

```


// TOPIC: list In STL

```

// 1. This is double linked list what we know from C programming language.
// 2. List is sequence container that allow non-contiguous memory allocation.
// 3. List is faster compared to other sequence containers (vector, forward_list, deque)
in terms of
//   insertion, removal and moving elements in any position provided we have the
iterator of the position.
// 4. We should use this class instead of traditional double linked list because
//   a. Well tested
//   b. Bunch of available function
// 5. Few Available Operations
//   operator =, assign, front, back, empty, size, max_size, clear, insert, emplace,
push_back, pop_back, push_front
//   pop_front, reverse, sort, merge, splice, unique, remove, remove_if, resize.

```

```
#include <iostream>
```

```
#include <list>
```

```
using namespace std;
```

```
int main()
{
    list<int> list1 = {5, 2, 4, 6, 2, 2, 2};
    list<int> list2 = {7, 6, 1, 9};
    list1.sort();
    list1.unique();
    for(auto& elm: list1)
        cout << elm << ' ';
    cout << endl;
    return 0;
}
```


// TOPIC: std::pair In C++

// SYNTAX:

// std::pair<T1, T2> obj;

// NOTES:

// 0. std::pair is a struct template that provides a way to store two heterogeneous objects as a single unit.

// 1. map, multimap, unordered_map, unordered_multimap can use pair to insert data into their structures.

`#include <iostream>`

`#include <vector>`

using namespace std;

```
void print(std::pair<int,int>& obj) { cout << obj.first << " " << obj.second << endl; }
```

```
int main()
```

```
{
```

```
{
```

```
    std::pair<int, int> obj(10, 20);
```

```
    print(obj);
```

```
}
```

```
{
```

```
    std::pair<int,int> obj = std::make_pair(10, 20);
```

```
    print(obj);
```

```
}
```

```
{
```

```
    std::vector<std::pair<std::string, uint>> list;
```

```
    list.push_back(make_pair("Rupesh", 30));
```

```
    list.push_back(make_pair("Hitesh", 28));
```

```
    list.push_back(std::pair<std::string,int>("Bhupendra", 22));
```

```
    list.push_back(std::pair("Sagar", 18));
```

```
    for(auto& elm: list) {
```

```
        cout << elm.first << " " << elm.second << endl;
```

```
    }
```

```
}
```

```
    return 0;
```

```
}
```

```
// TOPIC: deque In STL
```


// 1. deque is Double Ended Queue.

// 2. It is more similar to vector with few differences.

// a. vector internally contains elements in contiguous memory,
// where deque internally contains a list of memory chunks which store elements
contiguously.

// b. deque provides insertion at front, middle and end, where as vector provides at
middle and end only.

// c. vector performs good while insertion and deletion at end but bad for middle.

// d. deque performs good while inserting at front.

// e. No iterator invalidation happens in case of deque if insertion/deletion is from
middle.

// f. Random access operators like [] and at() are there in both but vector is faster
than deque.

// LINKS:

// 1. <https://stackoverflow.com/questions/6292332/what-really-is-a-deque-in-stl>
// 2. <https://baptiste-wicht.com/posts/2012/12/cpp-benchmark-vector-list-deque.html>

// NOTES:

// 1. Use deque when you have to do so much of insert/delete from front and back.
(Queue)

// 2. Use vector when you have to do so much of insert/delete from back. (Stack)

```
#include <iostream>
#include <deque>
using namespace std;

int main()
{
```

```
std::deque<int> dec(5);  
for(const auto& elm: dec)  
    cout << elm << endl;  
return 0;  
}
```


// TOPIC: std::queue In C++

// NOTES:

// 0. std::queue class is a container adapter that gives the programmer the functionality of a queue

// 1. queue is FIFO (first-in, first-out) data structure.

// 2. std::queue provides only specific set of functions.

// 3. std::queue allows to push(insert) at back and pop(remove) from front.

// 4. std::queue gives front, back, push, pop, empty, size.

// PROGRAM:

```
#include <iostream>
```

```
#include <queue>
```

```
using namespace std;
```

```
void print(std::queue<int> que) {  
    while(!que.empty()) {  
        cout << que.front() << endl;  
        que.pop(); // Remove from the front  
    }  
}
```

```
int main() {  
  
    std::queue<int> que;  
  
    que.push(2);  
    que.push(3);  
    que.push(4);  
  
    print(que);  
  
    return 0;  
}
```


// TOPIC: std::stack In C++ !

// NOTES:

// 0. std::stack class is a container adapter that gives the programmer the functionality of a stack.

// 1. Internally it uses std::deque STL container.

// 2. It is LIFO (last-in, first-out) data structure.

// 4. std::stack allows to push(insert) and pop(remove) only from back.

// FUNCTIONS PROVIDED:

// empty() – Returns whether the stack is empty – Time Complexity : $O(1)$

// size() – Returns the size of the stack – Time Complexity : $O(1)$

// top() – Returns a reference to the top most element of the stack – Time Complexity : $O(1)$

// push(g) – Adds the element 'g' at the top of the stack – Time Complexity : $O(1)$

// pop() – Deletes the top most element of the stack – Time Complexity : $O(1)$

// PROGRAM:

```
#include <iostream>
```

```
#include <stack>
```

```
#include <vector>
```

```
#include <set>
```

```
using namespace std;
```

```
void print(std::stack<int, std::set<int>> stk) {
```

```
    while(!stk.empty()) {
```

```
        cout << stk.top() << endl;
```

```
        stk.pop(); // Remove from the back
```

```
    }
```

```
}
```

```
int main() {
```

```
    std::stack<int, std::set<int>> stk;
```

```
    stk.push(2);
```

```
    stk.push(3);
```

```
    stk.push(4);
```

```
    print(stk);

    return 0;
}
```


// TOPIC: std::priority_queue In C++ (Priority Queue)

// WHAT IS IT ??

// NOTES:

// 0. std::priority_queue is a container adaptor that provides constant time lookup of the

// largest OR smallest element.

// 1. By default std::vector is the container used inside.

// 2. Cost of insertion and extraction is logarithmic.

// 3. std::priority_queue is implemented using std::make_heap, std::push_heap, std::pop_heap functions.

// PROGRAM

```
#include <functional>
```

```
#include <queue>
```

```
#include <vector>
```

```
#include <iostream>
```

```
template<typename T> void print_queue(T& q) {
    while(!q.empty()) {
```

```

        std::cout << q.top() << " ";
        q.pop();
    }
    std::cout << std::endl;
}

```

```

int main() {
    {
        std::priority_queue<int> q;
        for(int elm : {1,8,5,6,3,4,0,9,7,2}) { q.push(elm); }
        print_queue(q);
    }
    {
        std::priority_queue<int, std::vector<int>, std::greater<int> > q2;
        for(int elm : {1,8,5,6,3,4,0,9,7,2}) { q2.push(elm); }
        print_queue(q2);
    }
    // Using lambda to compare elements.
    {
        auto cmp = [](int left, int right) { return (left) < (right); };
        std::priority_queue<int, std::vector<int>, decltype(cmp)> q3(cmp);

        for(int elm : {1,8,5,6,3,4,0,9,7,2}) { q3.push(elm); }
        print_queue(q3);
    }
}

```

// TOPIC: std::unordered_set

// NOTES std::unordered_set:

// 0. Unorder Set is a associative container that contains set of unique objects.

// 1. Search, insertion, and removal have average constant-time complexity.

// 2. Internally, the elements are organized into buckets.

// 3. It uses hashing to insert elements into buckets.

// 4. This allows fast access to individual elements, since once a hash is computed,

// it refers to the exact bucket the element is placed into.

// WHY UNORDERED SET?

// 0. maintain a collection of unique items with fast insertion and removal.

// PROGRAM:

`#include <iostream>`

`#include <unordered_set>`

`using namespace std;`

`int main()`

`{`

`std::unordered_set<int> uset = {4, 1, 2, 3, 4, 2, 3};`

`auto search = uset.find(2);`

`if (search != uset.end()) {`

`std::cout << "Found " << (*search) << '\n';`

`} else {`

```

        std::cout << "Not found\n";
    }
    for(auto& elm: uset){
        std::cout << elm << " ";
    }
    cout << endl;
    return 0;
}

```


// TOPIC: std::unordered_multiset In C++!

// NOTES:

// 0. std::unordered_multiset is an associative container that contains set of possibly non-unique objects.

// 1. Search, insertion, and removal have average constant-time complexity.

// 2. Internally, the elements are organized into buckets.

// 3. It uses hashing to insert elements into buckets.

// 4. This allows fast access to individual elements, because after computing the hash of the value it refers to

// the exact bucket the element is placed into.

// WHY UNORDERED_MULTISET

// 0. maintain a collection of non-unique items with fast insertion and removal.

// PROGRAM:

```
#include <iostream>
```

```
#include <unordered_set>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    std::unordered_multiset<int> umset = {4, 1, 2, 3, 9, 4, 3, 2, 9, 8, 10};
```

```
    auto search = umset.find(2);
```

```
    if (search != umset.end()) {
```

```
        std::cout << "Found " << (*search) << '\n';
```

```
    } else {
```

```
        std::cout << "Not found\n";
```

```
    }
```

```
    for(auto& elm: umset){
```

```
        std::cout << elm << " ";
```

```
    }
```

```
    cout << endl;
```

```
    cout << umset.count(4) << endl;
```

```
    return 0;
```

```
}
```

// TOPIC: std::unordered_map In C++

// NOTES:

// 0. std::unordered_map is an associative container that contains key-value pairs with unique keys.

// 1. Search, insertion, and removal have average constant-time complexity.

```
// 2. Internally, the elements are organized into buckets.  
// 3. It uses hashing to insert elements into buckets.  
// 4. This allows fast access to individual elements, because after computing the hash  
of the value it refers to  
// the exact bucket the element is placed into.
```

```
// WHY UNORDERED_MAP
```

```
// 0. maintain a collection of unique {key:value} pairs with fast insertion and removal.
```

```
// PROGRAM:
```

```
#include <iostream>
```

```
#include <map>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    std::unordered_map<int,char> umap = {{1,'a'},{2,'b'}};
```

```
    // Access
```

```
    cout << umap[1] << endl;
```

```
    cout << umap[2] << endl;
```

```
    // Update
```

```
    umap[1] = 'c';
```

```
    // Iterate
```

```
    for(auto& elm: umap) {
```

```
        std::cout << elm.first << " " << elm.second << " ";
```

```
    }
```

```
    cout << endl;
```

```

// Find
auto result = umap.find(2);
if (result != umap.end()) {
    std::cout << "Found " << result->first << " " << result->second << '\n';
} else {
    std::cout << "Not found\n";
}

return 0;
}

```


// TOPIC: std::unordered_multimap In C++ STL!

// NOTES:

// 0. Unordered multimap is an unordered associative container that supports equivalent keys (an unordered_multimap

// may contain multiple copies of each key value) and that associates values of another type with the keys.

// 1. Search, insertion, and removal have average constant-time complexity.

// 2. Internally, the elements are organized into buckets.

// 3. It uses hashing to insert elements into buckets.

// 4. This allows fast access to individual elements, since once a hash is computed,

// it refers to the exact bucket the element is placed into.

// WHY UNORDERED_MULTIMAP

// 0. Maintain a collection of duplicate {key:value} pairs with fast insertion and removal.

// PROGRAM:

```
#include <iostream>
```

```
#include <unordered_map>
```

```
using namespace std;
```

```
int main() {
```

```
    std::unordered_multimap<int, char> umm = {{5,'d'}};
```

```
    umm.insert({1,'a'});
```

```
    umm.insert(std::pair<int, char>(2,'b'));
```

```
    umm.insert(std::make_pair(3,'c'));
```

```
    umm.insert(std::make_pair(3,'d'));
```

```
    for(auto& elm: umm) { cout << elm.first << " " << elm.second << endl; }
```

```
    return 0;
```

```
}
```


// TOPIC: STL Algorithms

// NOTES:

// 0. STL algorithms library defines functions for a variety of purposes (e.g. searching, sorting, counting, manipulating)

// 1. Algorithms are applied to range of elements.

// 3. <algorithm> header is used to get all algorithms in stl.

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int main() {
```

```
    std::vector<int> Vec{4,3,5,2,6,1};
```

```
    auto it = std::find(Vec.begin(), Vec.end(), 9);
```

```
    if(it == Vec.end()){
```

```
        cout << "Not Found" << endl;
```

```
    }
```

```
    else{
```

```
        cout << "Found" << endl;
```

```
    }
```

```
    return 0;
```

```
}
```


// TOPIC: STL Algorithm Sort

// NOTES:

// 0. In C++ STL we have sort function which can sort in increasing and decreasing order.

// 1. Not only integral but user defined data can be sorted using this function.

```
// 2. Internally it uses IntroSort which is combination of QuickSort, HeapSort and InsertionSort.  
  
// 3. By default, it uses QuickSort but if QuickSort is doing unfair partitioning and taking more than  $N \cdot \log N$  time,  
  
// it switches to HeapSort and when the array size becomes really small, it switches to InsertionSort.  
  
// 4. We can use parallel execution policy for better performance.
```

```
// TYPES:
```

```
// 1. Sorting integral data types  
// 2. Sorting user defined data types  
// 3. Sort using a function object  
// 4. Sort using lambda expression
```

```
#include <iostream>  
#include <algorithm>  
#include <vector>  
#include <execution>  
using namespace std;
```

```
// 1. Sort with integral data types
```

```
int main() {  
    std::vector<int> Vec{5,4,6,7,3,2,8,9,1};  
    std::sort(std::execution::par, Vec.begin(), Vec.end());  
    for(auto elm: Vec) {  
        cout << elm << " ";  
    }  
}
```

```
    return 0;
}
```

// 2. Sort with user defined data types

```
// class Point {
// public:
//     int x;
//     int y;
//     Point(int x=0, int y=0):x{x}, y{y} {}
//     bool operator < (const Point& p1) {
//         return (x+y)<(p1.x+p1.y);
//     }
// };

// int main() {
//     std::vector<Point> Vec {{1,2}, {3,1}, {0,1}};
//     std::sort(Vec.begin(), Vec.end());
//     for(auto e: Vec){
//         cout << e.x << " " << e.y << endl;
//     }
//     return 0;
// }
```

// 3. sort using a function object

```
// struct {
```

```

// bool operator()(int a, int b) const
// {
//     return a < b;
// }
// } customLess;

// int main() {
//     std::vector<int> Vec{5,4,6,7,3,2,8,9,1};
//     std::sort(Vec.begin(), Vec.end(), customLess);
//     for(auto elm: Vec) {
//         cout << elm << " ";
//     }
//     return 0;
// }

```

// 4. sort using lambda expression

```

// int main() {
//     std::vector<int> Vec{5,4,6,7,3,2,8,9,1};
//     std::sort(Vec.begin(), Vec.end(), std::less<int>());
//     for(auto elm: Vec) {
//         cout << elm << " ";
//     }
//     return 0;
// }

```

// TOPIC: STL Algorithm is_sort

// NOTES:

// 0. Checks if the elements in range [first, last) are sorted in non-descending order.

// TYPES:

// 1. Can check integral data types

// 2. Can check user defined data types

// 3. Can check using a function object

// 4. Can check using lambda expression

```
#include <iostream>
```

```
#include <algorithm>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main() {
```

```
    vector<int> Vec{5, 4, 6, 7, 3, 2, 8, 9, 1};
```

```
    sort(Vec.begin(), Vec.end(), greater<int>());
```

```
    for (auto elm : Vec) {
```

```
        cout << elm << " ";
```

```
    }
```

```
    cout << endl << is_sorted(Vec.begin(), Vec.end(), greater<int>());
```

```
    return 0;
```

```
}
```

```
-----  
-----
```

// TOPIC: STL Algorithm partial_sort

// NOTES:

// 0. partial_sort rearranges elements such that the range [first, middle) contains the sorted elements.

// 1. The order of equal elements is not guaranteed to be preserved.

// 2. The order of remaining elements is unspecified.

// Look for the overloads in cppref site.

```
#include <algorithm>
```

```
#include <functional>
```

```
#include <vector>
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    vector<int> Vec{1, 0, 7, 4, 2, 8, 6, 5, 9, 3};
```

```
    std::partial_sort(Vec.begin()+1, Vec.begin() + 4, Vec.end()-2, greater<int>());
```

```
    for (int val : Vec) {
```

```
        std::cout << val << " ";
```

```
    }
```

```
    return 0;
```

```
}
```

// TOPIC: STL Algorithms std::nth_element

// NOTES:

// 0. nth_element is a partial sorting algorithm that rearranges elements in [first, last) such that:

// a. The element at the nth position is the one which should be at that position if we sort the list.

// b. It does not sort the list, just that all the elements, which precede the nth element are not greater than it,

// and all the elements after it are not less than it.

// 1. nth_element algorithm is implemented using introselect.

// a. introselect is a hybrid of quickselect and median of medians algorithm.

// 1. quickselect is used to find kth smallest number in an unsorted array.

// 2. median of medians is a median selection algorithm for better pivot selection mainly used in quickselect.

// SUPPORTS TYPES:

// 1. Sorting integral data types

// 2. Sorting user defined data types

// 3. Sort using a function object

// 4. Sort using lambda expression

#include <iostream>

#include <vector>

#include <algorithm>

#include <functional>

int main() {

```
std::vector<int> v{5, 6, 4, 3, 2, 6, 7, 9, 3};
```

```
std::nth_element(v.begin(), v.begin() + v.size()/2, v.end());
```

```
std::cout << "The median is " << v[v.size()/2] << '\n';
```

```
std::nth_element(v.begin(), v.begin()+1, v.end(), std::greater<int>());
```

```
std::cout << "The second largest element is " << v[1] << '\n';
```

```
return 0;
```

```
}
```

```
// TOPIC: STL ALGORITHM is_sorted_until
```

```
// NOTES:
```

```
// 0. Finds how many elements are sorted in given range [first, last).
```

```
// SUPPORTS TYPES:
```

```
// 1. Sorting integral data types
```

```
// 2. Sorting user defined data types
```

```
// 3. Sort using a function object
```

```
// 4. Sort using lambda expression
```

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```

int main () {
    vector<int> Vec {4,3,2,1,7,8,1};
    auto it = is_sorted_until(Vec.begin()+1, Vec.end(), greater<int>());
    auto diff = distance(Vec.begin()+1, it);
    cout << *it << endl;
    cout << diff << endl;
    return 0;
}

```

// TOPIC: STL ALGORITHM std::stable_sort

// NOTES:

// 0. Sorts elements in the range [first, last) in ascending order. Order of equivalent elements is guaranteed to be preserved.

// 1. std::stable_sort usually uses mergesort for sorting.

// 2. This function attempts to allocate a temporary buffer equal in size to the sequence to be sorted.

// If the allocation fails, then the less efficient algorithm is chosen. (InsertionSort, BubbleSort)

// USAGE:

// 0. Sometime we want to make sure that order of equal elements is same in sorted array as it was in original array.

// This can be useful if these values have associated other fields. For example, consider sorting students by marks,

// SUPPORTS TYPES:

// 1. Sorting integral data types

// 2. Sorting user defined data types

```
// 3. Sort using a function object  
// 4. Sort using lambda expression
```

```
#include <algorithm>  
#include <iostream>  
#include <string>  
#include <vector>
```

```
struct MarksList{  
    int grade;  
    std::string name;  
};
```

```
bool operator<(const MarksList & lhs, const MarksList & rhs) {  
    return lhs.grade < rhs.grade;  
}
```

```
int main() {  
    std::vector<MarksList> v = { {'A', "Dave"}, {'B', "Alice"}, {'A', "Ken"}, {'B', "Eric"}, {'A',  
    "Carol"} };  
  
    std::sort(v.begin(), v.end());  
  
    for (const MarksList & e : v)  
        std::cout << e.grade << ", " << e.name << "\n";  
    return 0;  
}
```

```
-----  
-----
```

// TOPIC: STL ALGORITHM std::partial_sort_copy

// NOTES:

// 0. std::partial_sort is used for sorting the range within the entire container and copy to new container.

// 1. The main purpose of this container is to get the sorted data without changing the original container.

// 2. It returns an iterator pointing to the element that follows the last element written in the result sequence.

// 3. If source range > destination range then it copy till the size of destination.

// 4. If source range < destination range then it copy whole source to destination.

// SUPPORTS TYPES:

// 1. Sorting integral data types

// 2. Sorting user defined data types

// 3. Sort using a function object

// 4. Sort using lambda expression

// These types are explained into std::sort video please check if you don't know this !!

```
#include <algorithm>
```

```
#include <vector>
```

```
#include <functional>
```

```
#include <iostream>
```

```
int main() {
```

```
    std::vector<int> src{4, 2, 5, 1, 3, 9, 8, 10}, dst(4);
```

```
    std::vector<int>::iterator it;
```

```
    it = std::partial_sort_copy(src.begin(), src.end(), dst.begin(), dst.end());
```

```

std::cout << "Writing to the smaller vector in ascending order gives: ";
for (int a : dst) {
    std::cout << a << " ";
}
// Descending Order
it = std::partial_sort_copy(src.begin(), src.end(), dst.begin(), dst.end(),
std::greater<int>());

}

```

```

// TOPIC: STL ALGORITHM std::iota

```

```

// NOTES:

```

```

// 0. Fills the range [first, last) with sequentially increasing values,

```

```

//   starting with value and repetitively evaluating ++value.

```

```

#include <algorithm>

```

```

#include <iostream>

```

```

#include <list>

```

```

#include <numeric>

```

```

#include <vector>

```

```

using namespace std;

```

```

int main()

```

```

{

```

```

    std::list<int> lst(10);

```



```

std::iota(lst.begin(), lst.end(), -5);

for(auto elm: lst) cout << elm << endl;

std::vector<std::list<int>::iterator> Vec(lst.size());
std::iota(Vec.begin(), Vec.end(), lst.begin());

for(auto elm: Vec) cout << *elm << endl;

return 0;
}

```

// TOPIC: STL ALGORITHM copy and copy_if

// NOTES:

// 0. copy Copies the elements in the range, defined by [first, last), to another range beginning at d_first.

// 1. copy_if only copies if predicate returns true.

// 2. if source and destination range are overlapping then the behavior is undefined.

```
#include <algorithm>
```

```
#include <iostream>
```

```
#include <vector>
```

```
#include <iterator>
```

```
#include <numeric>
```

```
using namespace std;
```

```

void print(vector<int>& vec) {
    copy(vec.begin(), vec.end(), ostream_iterator<int>(cout, " "));
    cout << endl;
}

int main()
{
    vector<int> from_vector(10);
    iota(from_vector.begin(), from_vector.end(), 0);

    vector<int> to_vector1(from_vector.size());
    copy(from_vector.begin(), from_vector.end(), to_vector1.begin());
    print(to_vector1);

    // OR, alternatively,
    vector<int> to_vector2;
    copy(from_vector.begin(), from_vector.end(), back_inserter(to_vector2));
    print(to_vector2);

    // either way is equivalent to
    vector<int> to_vector = from_vector;

    // What is the benefits of copy function if we can do this..
    // Actually you can partially copy elements using copy() function.

    cout << "odd numbers in to_vector are: ";
    vector<int> to_vector3(5);
    copy_if(from_vector.begin(), from_vector.end(),
            to_vector3.begin(),

```

```
    [](int x) { return (x & 1); });  
    print(to_vector3);  
    return 0;  
}
```


// TOPIC: STL ALGORITHM copy_n and copy_backward

// NOTES:

// 0. std::copy_n() function is used to copy given number of elements from source to destination.

// 1. std::copy_backward() function copies the elements from the range, defined by [first, last),

// to another range ending at d_last. The elements are copied in reverse order (the last element is copied first),

// but their relative order is preserved.

```
#include <iostream>
```

```
#include <string>
```

```
#include <algorithm>
```

```
#include <iterator>
```

```
#include <vector>
```

```
#include <numeric>
```

```
using namespace std;
```

```
void print(vector<int>& vec) {  
    copy(vec.begin(), vec.end(), ostream_iterator<int>(cout, " "));  
    cout << endl;  
}
```

```
int main()
{

    vector<int> from_vector(10);
    iota(from_vector.begin(), from_vector.end(), 0);

    // COPY_N
    vector<int> to_vector1;
    copy_n(from_vector.begin(), 4, back_inserter(to_vector1));
    print(to_vector1);

    // COPY_BACKWARD
    vector<int> to_vector2(15);
    copy_backward(from_vector.begin(), from_vector.end(), to_vector2.end());
    print(to_vector2);

    return 0;
}
```
